MuleSoft Database & DataWeave
Advanced Concepts Handbook

with Use Cases & Complete Solutions

Enterprise Integration Learning Notes

Complete Study, Revision & Practice Guide

Every chapter includes **Use Cases with Full Solutions**
Real-Time Integration · DataWeave Operators · Collection Functions
String Functions · Advanced Patterns · Interview Q&A

Written like a classroom handbook – simple, practical, and exam-ready!

D Charan Jeet
Salesforce MuleSoft Developer – Cognizant

MuleSoft 4 · Anypoint Studio · DataWeave 2.0 · MySQL

© 2024 D Charan Jeet. Enterprise Integration Learning Series.

Contents

1 Real-Time MuleSoft Integration Concepts **3**
1.1 What is uhub-sapi? . . . 3
1.2 Auto-Generated Keys . . . 3
1.3 Stored Procedures – IN and OUT Parameters . . . 3
1.4 Correlation ID and Transaction ID . . . 4
1.5 Choice Router, Dynamic Queries, Connection Pooling . . . 4

2 DataWeave Operators **7**
2.1 Mathematical Operators . . . 7
2.2 Equality Operators . . . 7
2.3 Relational Operators . . . 7
2.4 Logical Operators . . . 7

3 DataWeave List Operators **10**
3.1 prepend – Add to Front . . . 10
3.2 append – Add to End . . . 10
3.3 remove – Remove Elements . . . 10

4 DataWeave Control Statements **13**
4.1 if-else . . . 13
4.2 else-if (Chained) . . . 13
4.3 if-only (Conditional Field) . . . 13

5 DataWeave Advanced Concepts **16**
5.1 Default Values . . . 16
5.2 Pattern Matching . . . 16
5.3 User-Defined Functions . . . 16
5.4 The do Operator . . . 16

6 DataWeave String Functions **20**
6.1 Complete String Function Reference . . . 20

7 Core Utility Functions **24**
7.1 sizeOf, typeOf, namesOf, keysOf, valuesOf, entriesOf . . . 24
7.2 indexOf and lastIndexOf . . . 24

8 DataWeave Core Collection Functions **27**
8.1 map . . . 27
8.2 mapObject . . . 27
8.3 filter . . . 27
8.4 filterObject . . . 27

9 Using map and filter Together **31**
9.1 Execution Order and Performance . . . 31
9.2 Nested map with filter . . . 31

10 Advanced Collection Operations **35**
10.1 flatten and flatMap 35
10.2 distinctBy, orderBy, groupBy, joinBy 35

11 Aggregation Functions – reduce **39**
11.1 What is reduce? 39

12 Utility Functions **43**
12.1 ++, –, pluck, to, uuid, zip, unzip 43

13 Quick Reference & Interview Cheat Sheet **47**
13.1 Complete Function Reference 47
13.2 Top 15 Interview Questions and Answers 47
13.3 Final Words 49

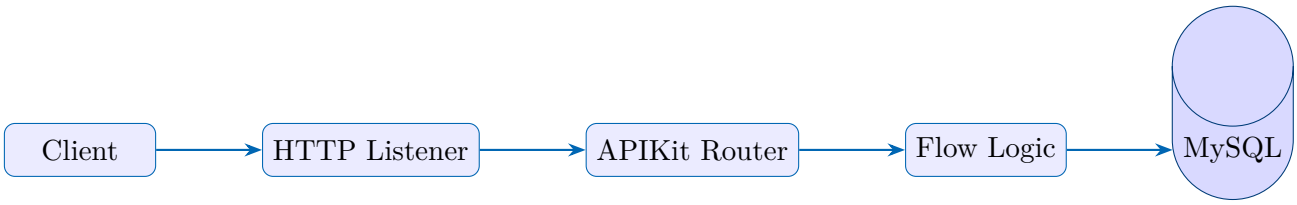
Chapter 1

Real-Time MuleSoft Integration Concepts

1.1 What is uhub-sapi?

In real MuleSoft projects, APIs follow the **3-Layer Architecture**:
Experience API → Process API → System API.

The **uhub-sapi** (Universal Hub System API) sits directly on top of the database. It is the API that does all the database heavy lifting – reading, writing, updating records.



1.2 Auto-Generated Keys

When you INSERT a record, MySQL auto-generates a primary key. The Database connector returns it in `payload.generatedKeys`.

```
-- MySQL table with AUTO_INCREMENT
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100),
    email       VARCHAR(100),
    city        VARCHAR(50)
);
```

```
// MuleSoft payload after INSERT
{
  "affectedRows": 1,
  "generatedKeys": [ { "GENERATED_KEY": 1042 } ]
}
```

1.3 Stored Procedures – IN and OUT Parameters

```
DELIMITER //
CREATE PROCEDURE register_order(
    IN  p_customer_id INT,
    IN  p_product_id  INT,
    IN  p_quantity    INT,
```

```
    OUT p_order_id     INT,
    OUT p_status       VARCHAR(50)
)
BEGIN
    INSERT INTO orders (customer_id, product_id, quantity,
       order_date)
    VALUES (p_customer_id, p_product_id, p_quantity, CURDATE());
    SET p_order_id = LAST_INSERT_ID();
    SET p_status   = 'ORDER_CREATED';
END //
DELIMITER ;
```

1.4 Correlation ID and Transaction ID

Concept	Description	Example Value
Correlation ID	Unique ID passed across all APIs for tracing	`UUID-9f3a2c`
Transaction ID	ID specific to one database transaction	`TXN-00123`

1.5 Choice Router, Dynamic Queries, Connection Pooling

Tip

Choice Router: Evaluates conditions top-to-bottom and routes to the first matching branch. Always include a Default route.

Dynamic Queries: Build the SQL string in DataWeave, pass user values as named parameters (`:param`) – never concatenate raw input.

Connection Pooling: Pre-creates and reuses database connections. Recommended: Min=5, Max=20 connections.

Use Case: Customer Registration API with Auto-Generated Key

Scenario: A retail application calls your MuleSoft API to register a new customer. After inserting the record, the API must return the newly generated `customer_id`.

Flow: HTTP Listener (POST /register) → DB Insert → Transform → HTTP Response

Input

```
// POST /register   (JSON body)
{
  "name":  "Rahul Verma",
  "email": "rahul@example.com",
  "city":  "Hyderabad"
}
```

Expected Output

```
{
  "message":    "Customer registered successfully",
  "customerId": 1042
}
```

Solution: Customer Registration API

Step 1 – Database Insert SQL

```
INSERT INTO customers (name, email, city)
VALUES (:name, :email, :city)
```

Step 2 – Input Parameters (DataWeave map in DB connector)

```
{
    name:  payload.name,
    email: payload.email,
    city:  payload.city
}
```

Step 3 – Transform Message (build response)

```
%dw 2.0
output application/json
---
{
    message:    "Customer registered successfully",
    customerId: payload.generatedKeys[0].GENERATED_KEY
}
```

Step 4 – Call Stored Procedure (alternative approach)

```
// Input Parameters for Stored Procedure call
{
    customer_id: { value: payload.customerId, type: "INTEGER"
        },
    product_id:  { value: payload.productId,  type: "INTEGER"
        },
    quantity:    { value: payload.quantity,    type: "INTEGER"
        },
    order_id:    { value: null, type: "INTEGER", inOut: "OUT"
        },
```

```
    status:      { value: null, type: "VARCHAR", inOut: "OUT"
        }
}
```

Step 5 – Read OUT parameters

```
%dw 2.0
output application/json
---
{
    orderId: payload.order_id,
    status:  payload.status
}
```

Step 6 – Dynamic Query with Choice Router

```
%dw 2.0
output application/java
---
// Build query based on optional city filter
{
    query: "SELECT * FROM customers"
        ++ (if (attributes.queryParams.city != null)
            " WHERE city = :city" else ""),
    params: if (attributes.queryParams.city != null)
        { city: attributes.queryParams.city }
    else {}
}
```

Interview Tip

Q: How does MuleSoft retrieve auto-generated keys from a DB insert?
A: The Database Insert operation returns `payload.generatedKeys[0].GENERATED_KEY` automatically. No extra SELECT query is needed.

Q: What is the difference between IN and OUT parameters in stored procedures?
A: IN passes data into the procedure; OUT returns data back to MuleSoft. Both are declared in the input parameters map using `inOut: "OUT"` for output parameters.

Chapter 2

DataWeave Operators

2.1 Mathematical Operators

```
%dw 2.0
output application/json
---
{
    addition:       10 + 5,       // 15
    subtraction:    10 - 5,       // 5
    multiplication: 10 * 5,       // 50
    division:       10 / 4,       // 2.5
    modulo:         10 mod 3,     // 1
    power:          2 ^ 8         // 256
}
```

2.2 Equality Operators

Operator	Meaning	Example
`==`	Strict equality	`5 == 5` → true
`!=`	Not equal	`5 != 6` → true
`~=`	Equal with type coercion	`5 ~= "5"` → true

2.3 Relational Operators

```
%dw 2.0
output application/json
---
{ gt: 10>5, lt: 10<5, gte: 10>=10, lte: 5<=10 }
// { "gt":true, "lt":false, "gte":true, "lte":true }
```

2.4 Logical Operators

```
%dw 2.0
output application/json
---
{
    andOp: (5>3) and (10>8),    // true
    orOp:  (5>3) or  (10<8),   // true
    notOp: not (5>3)            // false
}
```

Use Case: GST Invoice Calculator

Scenario: An e-commerce system sends a product payload. Your MuleSoft flow must calculate totals using mathematical and relational operators.

Input

```
{
  "productName": "Laptop",
  "basePrice":   50000,
  "quantity":    2,
  "gstRate":     18
}
```

Expected Output

```
{
  "productName":  "Laptop",
  "baseTotal":    100000,
  "gstAmount":    18000.0,
  "grandTotal":   118000.0,
  "isPremium":    true,
  "discountNote": "5% loyalty discount applied"
}
```

Solution: GST Invoice Calculator

```
%dw 2.0
output application/json
---
do {
    var baseTotal  = payload.basePrice * payload.quantity
    var gstAmount  = baseTotal * (payload.gstRate / 100)
    var grandTotal = baseTotal + gstAmount
    var isPremium  = baseTotal > 50000
    ---
    {
        productName:  payload.productName,
        baseTotal:    baseTotal,
        gstAmount:    gstAmount,
        grandTotal:   grandTotal,
        isPremium:    isPremium,
        // Only show discountNote if isPremium is true
        (discountNote: "5% loyalty discount applied") if
            isPremium
    }
}
```

Use Case: Student Eligibility Checker

Scenario: Check if a student is eligible for a scholarship using logical and equality operators.

Input

```
{ "name":"Arjun", "marks":82, "attendance":88, "category
    ":"merit" }
```

Expected Output

```
{ "student":"Arjun", "eligible":true, "reason":"Meets all
     criteria" }
```

Solution: Student Eligibility Checker

```
%dw 2.0
output application/json
---
do {
    var marksOk      = payload.marks >= 75
    var attendanceOk = payload.attendance >= 85
    var categoryOk   = payload.category == "merit"
    var eligible     = marksOk and attendanceOk and categoryOk
    ---
    {
        student:  payload.name,
        eligible: eligible,
        reason:   if (eligible) "Meets all criteria"
            else if (not marksOk)      "Marks below 75"
            else if (not attendanceOk) "Attendance below 85%"
            else                       "Category mismatch"
    }
}
```

Interview Tip

Q: What is the difference between == and ~= in DataWeave?
A: `==` requires both value and type to match. `~=` allows type coercion – `5 ~= "5"` is true.

Chapter 3

DataWeave List Operators

3.1 prepend – Add to Front

```
// >> adds element to BEGINNING of array
1 >> [2, 3, 4]        // [1, 2, 3, 4]
"NEW" >> ["A","B"]    // ["NEW","A","B"]
```

3.2 append – Add to End

```
// << adds element to END of array
[1, 2, 3] << 4        // [1, 2, 3, 4]
["A","B"] << "NEW"    // ["A","B","NEW"]
```

3.3 remove – Remove Elements

```
// - removes ALL occurrences of a value
[1, 2, 3, 2, 4] - 2   // [1, 3, 4]

// -- removes a sub-array of values
[1, 2, 3, 4, 5] -- [2, 4]  // [1, 3, 5]
```

Use Case: Shopping Cart Manager

Scenario: An e-commerce cart API receives operations to add items to the front (express lane), add to the end (regular), or remove cancelled items from a shopping cart.

Input

```
{
  "cart":       ["Mouse","Keyboard","Monitor"],
  "addFront":   "Laptop",
  "addEnd":     "USB Hub",
  "removeItem":"Keyboard"
}
```

Expected Output

```
{
  "withFrontItem": ["Laptop","Mouse","Keyboard","Monitor
     "],
  "withEndItem":   ["Mouse","Keyboard","Monitor","USB Hub
     "],
  "afterRemoval":  ["Mouse","Monitor"]
}
```

Solution: Shopping Cart Manager

```
%dw 2.0
output application/json
---
{
    withFrontItem: payload.addFront >> payload.cart,
    withEndItem:   payload.cart << payload.addEnd,
    afterRemoval:  payload.cart - payload.removeItem
}
```

Use Case: Notification Queue Builder

Scenario: A notification system receives an existing queue, a new URGENT message that must go to the front, routine messages to append, and expired items to remove.

Input

```
{
  "queue":        ["WELCOME_MSG","PROMO_MSG","SURVEY_MSG
     "],
  "urgentMsg":    "SYSTEM_ALERT",
  "routineMsg":   "NEWSLETTER",
  "expiredMsgs":  ["PROMO_MSG","SURVEY_MSG"]
}
```

Expected Output

```
{
  "updatedQueue": ["SYSTEM_ALERT","WELCOME_MSG","
     NEWSLETTER"]
}
```

Solution: Notification Queue Builder

```
%dw 2.0
output application/json
---
do {
    // Step 1: remove expired messages
    var cleaned = payload.queue -- payload.expiredMsgs
    // Step 2: prepend urgent
    var withUrgent = payload.urgentMsg >> cleaned
    // Step 3: append routine
    var finalQueue = withUrgent << payload.routineMsg
    ---
    {
        updatedQueue: finalQueue
    }
}
```

Interview Tip

Q: What is the difference between prepend and append in DataWeave?
A: » (prepend) inserts at index 0. « (append) adds at the last index. Both return new arrays without mutating the original.

Chapter 4

DataWeave Control Statements

4.1 if-else

```
// Basic if-else
if (payload.marks >= 40) "Pass" else "Fail"
```

4.2 else-if (Chained)

```
if      (payload.marks >= 90) "A+"
else if (payload.marks >= 75) "A"
else if (payload.marks >= 60) "B"
else if (payload.marks >= 40) "C"
else                          "Fail"
```

4.3 if-only (Conditional Field)

```
// Field only appears if condition is true
{
    name: payload.name,
    (discount: "10%") if (payload.city == "Hyderabad")
}
```

Use Case: Bank Loan Eligibility API

Scenario: A banking system checks loan eligibility based on salary, credit score, and existing loans. Return approval status, max loan amount, and interest rate. Add a `fastTrack` field only for premium customers.

Input

```
{
  "applicantName": "Ravi Kumar",
  "salary":        75000,
  "creditScore":   780,
  "existingLoans": 0,
  "customerType":  "premium"
}
```

Expected Output

```
{
  "applicant":      "Ravi Kumar",
  "status":         "APPROVED",
  "maxLoanAmount": 3000000,
  "interestRate":   "8.5%",
  "fastTrack":      true
}
```

Solution: Bank Loan Eligibility API

```
%dw 2.0
output application/json
---
do {
    var eligible = payload.creditScore >= 700
        and payload.salary >= 50000
        and payload.existingLoans == 0

    var maxLoan = if (payload.salary >= 100000)  5000000
        else if (payload.salary >= 75000)         3000000
        else if (payload.salary >= 50000)         1500000
        else                                       500000

    var rate = if (payload.creditScore >= 800)    "7.5%"
        else if (payload.creditScore >= 750)       "8.5%"
        else if (payload.creditScore >= 700)       "10.0%"
        else                                       "12.0%"

    var isPremium = payload.customerType == "premium"
    ---
    {
        applicant:     payload.applicantName,
        status:        if (eligible) "APPROVED" else "REJECTED
            ",
        maxLoanAmount: if (eligible) maxLoan else 0,
        interestRate:  if (eligible) rate else "N/A",
        (fastTrack: true) if (eligible and isPremium)
    }
}
```

Use Case: Delivery Zone Classifier

Scenario: Classify delivery orders into zones and assign estimated delivery days based on the destination city.

Input

```
{ "orderId":"ORD-001", "city":"Hyderabad", "weight":3.5 }
```

Expected Output

```
{
  "orderId":        "ORD-001",
  "zone":           "Zone A",
  "deliveryDays":   2,
  "expressOption":  true
}
```

Solution: Delivery Zone Classifier

```
%dw 2.0
output application/json
---
do {
    var zoneA = ["Hyderabad","Bangalore","Chennai","Mumbai","
       Delhi"]
    var zoneB = ["Pune","Kochi","Jaipur","Lucknow"]

    var zone = if (zoneA contains payload.city)        "Zone A"
        else if (zoneB contains payload.city)           "Zone B
           "
        else                                             "Zone
           C"

    var days = if (zone == "Zone A") 2
        else if (zone == "Zone B")   4
        else                         7
    ---
    {
        orderId:        payload.orderId,
        zone:           zone,
        deliveryDays:   days,
        (expressOption: true) if (zone == "Zone A")
    }
}
```

Interview Tip

Q: What is if-only in DataWeave?
A: It conditionally includes a field in an object. The field is absent (not null) when the condition is false. Syntax: `(fieldName: value) if (condition)`.

Chapter 5

DataWeave Advanced Concepts

5.1 Default Values

```
{
    city:    payload.city    default "Unknown",
    country: payload.country default "India",
    age:     payload.age     default 0
}
```

5.2 Pattern Matching

```
{
    category: payload.code match {
        case "A" -> "Electronics"
        case "B" -> "Clothing"
        case "C" -> "Food"
        else     -> "Unknown"
    }
}
```

5.3 User-Defined Functions

```
%dw 2.0
output application/json
fun addGST(price: Number, rate: Number) = price + (price * rate /
    100)
fun fullName(first: String, last: String) = upper(first) ++ " " ++
    upper(last)
---
{
    price:    addGST(50000, 18),
    name:     fullName("ravi", "kumar")
}
```

5.4 The do Operator

```
// do creates a local variable scope
payload.items map (item) ->
    do {
        var base     = item.qty * item.price
        var tax      = base * 0.18
        var total    = base + tax
        ---
        { item: item.name, total: total }
```

```
}
```

Use Case: Order Summary Enrichment with Defaults

Scenario: Orders arrive from multiple channels. Some fields may be missing. Enrich each order using defaults, pattern matching for order type, and a reusable discount function.

Input

```
{
  "orders": [
    { "id":"01","type":"B2B","amount":200000,"customer":"
       Infosys" },
    { "id":"02","type":"B2C","amount":4500 },
    { "id":"03","amount":12000,"customer":"Ravi" }
  ]
}
```

Expected Output

```
[
  {"orderId":"01","customer":"Infosys","type":"Business",
   "amount":200000,"discount":10000.0,"finalAmount
      ":190000.0},
  {"orderId":"02","customer":"Walk-in","type":"Consumer",
   "amount":4500,"discount":0.0,"finalAmount":4500.0},
  {"orderId":"03","customer":"Ravi","type":"General",
   "amount":12000,"discount":600.0,"finalAmount":11400.0}
]
```

Solution: Order Summary Enrichment with Defaults

```
%dw 2.0
output application/json

// Reusable discount function: returns discount amount
fun calcDiscount(amount: Number, orderType: String) =
    amount * (
        if      (orderType == "B2B") 0.05
        else if (orderType == "B2C") 0.00
        else                         0.05
    )

// Reusable label mapper using pattern matching
fun orderLabel(code: String) = code match {
    case "B2B" -> "Business"
    case "B2C" -> "Consumer"
    else       -> "General"
}
```

```
---
payload.orders map (order) ->
    do {
        var typ      = order.type     default "GENERAL"
        var cust     = order.customer default "Walk-in"
        var amt      = order.amount   default 0
        var disc     = calcDiscount(amt, typ)
        var final    = amt - disc
        ---
        {
            orderId:     order.id,
            customer:    cust,
            type:        orderLabel(typ),
            amount:      amt,
            discount:    disc,
            finalAmount: final
        }
    }
```

Use Case: Shipment Type Classifier with Pattern Matching

Scenario: Classify shipments by their carrier code and assign SLA days using pattern matching.

Input

```
{
  "shipments": [
    {"trackId":"T1","carrier":"FDX","weight":2.5},
    {"trackId":"T2","carrier":"DHL","weight":12.0},
    {"trackId":"T3","carrier":"IND","weight":0.8},
    {"trackId":"T4","carrier":"XYZ","weight":4.0}
  ]
}
```

Expected Output

```
[
  {"trackId":"T1","carrier":"FedEx","slaDays":2,"priority
     ":"Standard"},
  {"trackId":"T2","carrier":"DHL",  "slaDays":3,"priority
     ":"Standard"},
  {"trackId":"T3","carrier":"India Post","slaDays":7,"
     priority":"Economy"},
  {"trackId":"T4","carrier":"Unknown","slaDays":10,"
     priority":"Economy"}
]
```

Solution: Shipment Classifier with Pattern Matching

```
%dw 2.0
output application/json

fun carrierName(code: String) = code match {
    case "FDX" -> "FedEx"
    case "DHL" -> "DHL"
    case "IND" -> "India Post"
    else       -> "Unknown"
}

fun slaDays(code: String) = code match {
    case "FDX" -> 2
    case "DHL" -> 3
    case "IND" -> 7
    else       -> 10
}
---
payload.shipments map (s) -> {
    trackId:  s.trackId,
    carrier:  carrierName(s.carrier),
    slaDays:  slaDays(s.carrier),
    priority: if (slaDays(s.carrier) <= 3) "Standard" else "
       Economy"
}
```

Interview Tip

Q: When would you use the `do` operator in DataWeave?
A: When you need local intermediate variables within a `map` or complex expression. It keeps code readable by naming computed values instead of nesting expressions.

Q: How do user-defined functions help in DataWeave?
A: They keep transformations DRY. Define once at the top with `fun`, reuse anywhere in the script.

Chapter 6

DataWeave String Functions

6.1 Complete String Function Reference

```
%dw 2.0
output application/json
---
{
    upper:        upper("hello world"),          // "HELLO WORLD"
    lower:        lower("HELLO WORLD"),          // "hello world"
    capitalize:   capitalize("hello world"),     // "Hello world"
    trim:         trim("  hello  "),             // "hello"
    ltrim:        ltrim("  hello"),              // "hello"
    rtrim:        rtrim("hello  "),              // "hello"
    replaced:     replace("hello world","world") with "DataWeave",
    startsWith:   "mulesoft" startsWith "mule", // true
    endsWith:     "mulesoft" endsWith "soft",   // true
    contains:     "integration" contains "grate", // true
    split:        "a,b,c" splitBy ",",          // ["a","b","c"]
    joined:       ["a","b","c"] joinBy ",",     // "a,b,c"
    strLen:       sizeOf("hello"),              // 5
    indexOf:      "hello" indexOf "l",          // 2
    lastIndexOf:  "hello" lastIndexOf "l",      // 3
    substring:    "Hello World"[0 to 4],        // "Hello"
    concat:       "Hello" ++ " " ++ "World"     // "Hello World"
}
```

Use Case: Customer Data Normalisation API

Scenario: Customer records arrive from a legacy CRM with inconsistent casing, extra whitespace, and full names as a single field. Normalise the data before storing in the database.

Input

```
{
  "customers": [
    { "fullName":"  ravi kumar  ","email":"RAVI@GMAIL.COM
      ","city":"hyderabad" },
    { "fullName":"PRIYA SHARMA", "email":"priya@yahoo.com
      ","city":"CHENNAI"  },
    { "fullName":"Arjun  Reddy","email":"arjun@hotmail.
      com","city":"bangalore"}
  ]
}
```

Expected Output

```
[
  {"firstName":"Ravi","lastName":"Kumar",
   "email":"ravi@gmail.com","city":"Hyderabad"},
  {"firstName":"Priya","lastName":"Sharma",
   "email":"priya@yahoo.com","city":"Chennai"},
  {"firstName":"Arjun","lastName":"Reddy",
   "email":"arjun@hotmail.com","city":"Bangalore"}
]
```

Solution: Customer Data Normalisation API

```
%dw 2.0
output application/json
---
payload.customers map (c) ->
    do {
        // trim and normalise the full name
        var cleanName = trim(c.fullName)
        var parts     = cleanName splitBy " "
            filter ($ != "")   // remove empty parts from
               double spaces
        var firstName = capitalize(lower(parts[0] default ""))
        var lastName  = capitalize(lower(parts[1] default ""))
        ---
        {
            firstName: firstName,
            lastName:  lastName,
            email:     lower(trim(c.email)),
            city:      capitalize(lower(trim(c.city)))
        }
    }
```

Use Case: Product SKU Generator

Scenario: Generate a standardised SKU code for each product by combining category prefix, product name (first 4 chars, uppercase), and price tier.

Input

```
{
  "products": [
    {"name":"Wireless Mouse","category":"Electronics","
       price":999},
    {"name":"Running Shoes", "category":"Footwear",   "
       price":4500},
    {"name":"Notebook Set",  "category":"Stationery", "
       price":250}
  ]
}
```

Expected Output

```
[
  {"product":"Wireless Mouse","sku":"ELE-WIRE-BUDGET"},
  {"product":"Running Shoes", "sku":"FOO-RUNN-MID"},
  {"product":"Notebook Set",  "sku":"STA-NOTE-BUDGET"}
]
```

Solution: Product SKU Generator

```
%dw 2.0
output application/json

fun categoryPrefix(cat: String) =
    upper(cat[0 to 2])   // first 3 chars of category

fun namePart(n: String) =
    upper(trim(n)[0 to 3])  // first 4 chars of name

fun priceTier(p: Number) =
    if      (p > 10000) "PREMIUM"
    else if (p > 2000)  "MID"
    else                "BUDGET"
---
payload.products map (p) -> {
    product: p.name,
    sku:     categoryPrefix(p.category) ++ "-"
          ++ namePart(p.name) ++ "-"
          ++ priceTier(p.price)
}
```

Interview Tip

Q: How do you split a full name string into first and last name in DataWeave?
A: Use `splitBy " "` to get an array, then index into it: `(name splitBy "`

`")[0]` for first name, `[1]` for last name.

Chapter 7

Core Utility Functions

7.1 sizeOf, typeOf, namesOf, keysOf, valuesOf, entriesOf

```
%dw 2.0
output application/json
var customer = { id: 1, name: "Ravi", city: "Hyderabad" }
---
{
    sizeOfArray:  sizeOf([1,2,3,4]),          // 4
    sizeOfStr:    sizeOf("hello"),            // 5
    sizeOfObj:    sizeOf(customer),           // 3

    typeNum:      typeOf(42),                 // "Number"
    typeStr:      typeOf("hello"),            // "String"
    typeBool:     typeOf(true),               // "Boolean"
    typeArr:      typeOf([1,2]),              // "Array"
    typeObj:      typeOf({a:1}),              // "Object"

    keys:         keysOf(customer),           // ["id","name","city
       "]
    values:       valuesOf(customer),         // [1,"Ravi","
       Hyderabad"]
    entries:      entriesOf(customer)         // [{key:"id",value
       :1},...]
}
```

7.2 indexOf and lastIndexOf

```
%dw 2.0
output application/json
---
{
    firstL:  "hello" indexOf "l",          // 2
    lastL:   "hello" lastIndexOf "l",      // 3
    arrIdx:  [1,2,3,2,4] indexOf 2,        // 1
    lastIdx: [1,2,3,2,4] lastIndexOf 2     // 3
}
```

Use Case: Payload Inspection and Schema Validation

Scenario: Before saving an order to the database, validate that all required fields are present and have the correct types using utility functions.

Input

```
{
  "orderId":     "ORD-2024-001",
  "customerId": 42,
  "items":       ["Laptop","Mouse"],
  "amount":      87500
}
```

Expected Output

```
{
  "valid":           true,
  "fieldCount":      4,
  "detectedTypes": {
    "orderId":"String","customerId":"Number",
    "items":"Array","amount":"Number"
  },
  "itemCount":       2,
  "allFieldsPresent": true
}
```

Solution: Payload Inspection and Schema Validation

```
%dw 2.0
output application/json
---
do {
    var requiredFields = ["orderId","customerId","items","
        amount"]
    var payloadKeys    = keysOf(payload) map ($ as String)
    var allPresent     = (requiredFields -- payloadKeys) == []
    ---
    {
        valid:           allPresent,
        fieldCount:      sizeOf(payload),
        detectedTypes:   payload mapObject (value, key) ->
                             { (key): typeOf(value) },
        itemCount:       sizeOf(payload.items),
        allFieldsPresent: allPresent
    }
}
```

Use Case: Dynamic Header Builder using entriesOf

Scenario: Convert an object of HTTP headers into an array of key-value pair strings for logging, and find the position of a specific header.

Input

```
{
  "headers": {
    "Content-Type":  "application/json",
    "Authorization": "Bearer abc123",
    "X-Request-ID":  "REQ-001",
    "Accept":        "application/json"
  }
}
```

Expected Output

```
{
  "headerList":  ["Content-Type: application/json",
                  "Authorization: Bearer abc123",
                  "X-Request-ID: REQ-001",
                  "Accept: application/json"],
  "headerCount": 4,
  "authIndex":   1
}
```

Solution: Dynamic Header Builder using entriesOf

```
%dw 2.0
output application/json
---
do {
    var entries    = entriesOf(payload.headers)
    var headerList = entries map (e) -> (e.key as String) ++
       ": " ++ e.value
    var keys       = keysOf(payload.headers) map ($ as String)
    ---
    {
        headerList:  headerList,
        headerCount: sizeOf(payload.headers),
        authIndex:   keys indexOf "Authorization"
    }
}
```

Interview Tip

Q: What is the difference between keysOf and namesOf?
A: They return the same result – an array of key names from an object. `namesOf` is an alias for `keysOf`.

Q: How would you check if a field exists in a payload?
A: `keysOf(payload) contains "fieldName"` or simply `payload.fieldName != null`.

Chapter 8

DataWeave Core Collection Functions

8.1 map

```
// map transforms EVERY element -- same count in, same count out
payload.students map (student, index) -> {
    position:    index + 1,
    studentName: student.name,
    result:      if (student.marks >= 40) "Pass" else "Fail"
}
```

8.2 mapObject

```
// mapObject transforms key-value pairs of an OBJECT
{ laptop: 50000, mouse: 500 } mapObject (value, key) -> {
    (key): value * 1.18   // add 18% GST to every price
}
```

8.3 filter

```
// filter keeps only elements where condition is TRUE
payload.students filter ($.marks >= 40)       // only passing
   students
payload.products filter ($.price > 1000)      // only expensive
   products
```

8.4 filterObject

```
// filterObject removes unwanted keys from an object
{ name:"Ravi", age:25, city:null, phone:null }
filterObject (value, key) -> value != null
// Result: { name:"Ravi", age:25 }
```

Use Case: Employee Payroll Processor

Scenario: Process employee records to calculate net pay, filter only active employees, and transform the config object by applying a tax multiplier to each department budget.

Input

```
{
  "employees": [
    {"id":1,"name":"Ravi",  "salary":80000,"status":"
       active","dept":"Tech"},
    {"id":2,"name":"Priya", "salary":65000,"status":"
       inactive","dept":"HR"},
    {"id":3,"name":"Arjun", "salary":95000,"status":"
       active","dept":"Tech"},
    {"id":4,"name":"Kiran", "salary":55000,"status":"
       active","dept":"Sales"}
  ],
  "deptBudgets": { "Tech":500000, "HR":200000, "Sales
     ":300000 }
}
```

Expected Output

```
{
  "payroll": [
    {"id":1,"name":"Ravi", "grossSalary":80000,"tax
       ":8000.0,"netPay":72000.0},
    {"id":3,"name":"Arjun","grossSalary":95000,"tax
       ":9500.0,"netPay":85500.0},
    {"id":4,"name":"Kiran","grossSalary":55000,"tax
       ":5500.0,"netPay":49500.0}
  ],
  "budgetsAfterTax": {"Tech":450000.0,"HR":180000.0,"
     Sales":270000.0}
}
```

Solution: Employee Payroll Processor

```
%dw 2.0
output application/json
---
{
    // Step 1: filter active, then map to add tax and net pay
    payroll: payload.employees
        filter ($.status == "active")
        map (emp) ->
            do {
                var tax    = emp.salary * 0.10
                var netPay = emp.salary - tax
                ---
                {
                    id:           emp.id,
                    name:         emp.name,
                    grossSalary:  emp.salary,
                    tax:          tax,
                    netPay:       netPay
```

```
                }
            },

    // Step 2: mapObject to apply 10% tax deduction to each
       dept budget
    budgetsAfterTax: payload.deptBudgets
        mapObject (budget, dept) -> {
            (dept): budget * 0.90
        }
}
```

Use Case: API Response Sanitiser

Scenario: A third-party API returns a response full of null fields and unwanted metadata. Remove all null values and internal fields (those starting with "_").

Input

```
{
  "_meta": "internal", "_version": "2.1",
  "customerId": 101,
  "name": "Ravi Kumar",
  "email": null,
  "phone": null,
  "city": "Hyderabad"
}
```

Expected Output

```
{ "customerId":101, "name":"Ravi Kumar", "city":"
    Hyderabad" }
```

Solution: API Response Sanitiser

```
%dw 2.0
output application/json
---
payload
    // Step 1: remove null values
    filterObject (value, key) -> value != null
    // Step 2: remove internal fields starting with "_"
    filterObject (value, key) ->
        not ((key as String) startsWith "_")
```

Interview Tip

Q: What is the difference between map and mapObject?
A: `map` works on **arrays** and returns an array. `mapObject` works on **objects**

(key-value pairs) and returns an object.

Q: What is the difference between filter and filterObject?
A: `filter` removes elements from an **array**. `filterObject` removes key-value pairs from an **object**.

Chapter 9

Using map and filter Together

9.1 Execution Order and Performance

Always **filter first, then map**. This reduces the number of items being transformed.

```
// GOOD: filter first, then map (processes only matching items)
payload.products
    filter ($.price > 1000)
    map (item) -> {
        productName:  item.name,
        priceWithGST: item.price * 1.18
    }

// AVOID: map first (transforms ALL items before filtering)
payload.products
    map (item) -> { productName: item.name, priceWithGST: item.
       price * 1.18 }
    filter ($.priceWithGST > 1180)
```

9.2 Nested map with filter

```
payload.orders
    filter ($.status == "PENDING")
    map (order) -> {
        orderId:    order.id,
        highValue:  order.items filter ($.price > 5000) map ($.name
           )
    }
```

Use Case: Premium Order Report Generator

Scenario: From a list of orders, generate a report of only high-value COMPLETED orders (amount over 50000), showing customer name, total with GST, and flagging VIP customers.

Input

```
{
  "orders": [
    {"id":"01","customer":"Infosys","amount":200000,
     "status":"COMPLETED","isVip":true},
    {"id":"02","customer":"Ravi",   "amount":4500,
     "status":"COMPLETED","isVip":false},
    {"id":"03","customer":"TCS",    "amount":150000,
     "status":"PENDING","isVip":true},
    {"id":"04","customer":"Wipro",  "amount":85000,
     "status":"COMPLETED","isVip":false},
    {"id":"05","customer":"Priya",  "amount":75000,
     "status":"COMPLETED","isVip":true}
  ]
}
```

Expected Output

```
{
  "reportCount": 3,
  "orders": [
    {"orderId":"01","customer":"Infosys","total
       ":236000.0,"vipFlag":"VIP"},
    {"orderId":"04","customer":"Wipro",  "total
       ":100300.0,"vipFlag":""},
    {"orderId":"05","customer":"Priya",  "total":88500.0,
        "vipFlag":"VIP"}
  ]
}
```

Solution: Premium Order Report Generator

```
%dw 2.0
output application/json
---
do {
    // Step 1: filter -- only completed orders over 50000
    var filtered = payload.orders
        filter ($.status == "COMPLETED" and $.amount > 50000)

    // Step 2: map -- transform each qualifying order
    var report = filtered map (o) -> {
        orderId:  o.id,
        customer: o.customer,
        total:    o.amount * 1.18,
        vipFlag:  if (o.isVip) "VIP" else ""
    }
    ---
    {
        reportCount: sizeOf(report),
        orders:      report
```

```
    }
}
```

Use Case: Nested Order Item Processor

Scenario: Each order contains items. Filter orders that are not cancelled, then for each order filter items above a minimum value and return the item names in uppercase.

Input

```
{
  "orders": [
    {"id":"01","status":"ACTIVE",
     "items":[{"name":"Laptop","price":85000},{"name":"
        Mouse","price":500}]},
    {"id":"02","status":"CANCELLED",
     "items":[{"name":"Keyboard","price":3500}]},
    {"id":"03","status":"ACTIVE",
     "items":[{"name":"Monitor","price":22000},{"name":"
        Pen","price":50}]}
  ]
}
```

Expected Output

```
[
  {"orderId":"01","premiumItems":["LAPTOP"]},
  {"orderId":"03","premiumItems":["MONITOR"]}
]
```

Solution: Nested Order Item Processor

```
%dw 2.0
output application/json
---
payload.orders
    filter ($.status != "CANCELLED")
    map (order) -> {
        orderId:       order.id,
        premiumItems: order.items
            filter  ($.price > 1000)
            map     (item) -> upper(item.name)
    }
```

Interview Tip

Q: Why should you filter before map in DataWeave?
A: `filter` reduces the array size first, so `map` only processes qualifying elements. This improves performance, especially on large datasets from database queries.

Chapter 10

Advanced Collection Operations

10.1 flatten and flatMap

```
// flatten: [[1,2],[3,4]] -> [1,2,3,4]
flatten([[1,2],[3,4],[5,6]])     // [1,2,3,4,5,6]

// flatMap: map then auto-flatten
payload.orders flatMap (o) -> o.items
// If orders=[{items:["A","B"]},{items:["C","D"]}] -> ["A","B","C
   ","D"]
```

10.2 distinctBy, orderBy, groupBy, joinBy

```
// distinctBy: remove duplicates
[{city:"HYD"},{city:"BLR"},{city:"HYD"}] distinctBy $.city
// -> [{city:"HYD"},{city:"BLR"}]

// orderBy: sort ascending (negate for descending)
payload.products orderBy $.price
payload.products orderBy -$.price    // descending

// groupBy: group into object keyed by field
payload.customers groupBy $.city
// -> { "HYD":[...], "BLR":[...] }

// joinBy: array to string
["A","B","C"] joinBy ","   // "A,B,C"
```

Use Case: Sales Analytics Dashboard Data Builder

Scenario: Build a sales analytics payload from raw order data: extract all unique cities, get a flat list of all product names across all orders, group orders by city, and produce a sorted leaderboard.

Input

```
{
  "orders": [
    {"id":"01","city":"Hyderabad","amount":85000,
     "items":["Laptop","Mouse"]},
    {"id":"02","city":"Bangalore","amount":22000,
     "items":["Monitor","Keyboard"]},
    {"id":"03","city":"Hyderabad","amount":4500,
     "items":["USB Hub"]},
    {"id":"04","city":"Chennai",  "amount":95000,
     "items":["Laptop","Charger"]},
    {"id":"05","city":"Bangalore","amount":12000,
     "items":["Mouse","Pen"]}
  ]
}
```

Expected Output

```
{
  "uniqueCities":   ["Hyderabad","Bangalore","Chennai"],
  "allProducts":    ["Laptop","Mouse","Monitor","Keyboard
     ",
                     "USB Hub","Laptop","Charger","Mouse
                        ","Pen"],
  "ordersByCity": {
    "Hyderabad": [...], "Bangalore": [...], "Chennai":
       [...]
  },
  "topOrdersSorted": [
    {"id":"04","amount":95000},
    {"id":"01","amount":85000},
    {"id":"02","amount":22000},
    {"id":"05","amount":12000},
    {"id":"03","amount":4500}
  ]
}
```

Solution: Sales Analytics Dashboard Data Builder

```
%dw 2.0
output application/json
---
{
    // distinctBy to get unique cities
    uniqueCities: (payload.orders map $.city) distinctBy $,

    // flatMap to flatten all items arrays into one list
    allProducts:  payload.orders flatMap $.items,

    // groupBy to group orders by city
    ordersByCity: payload.orders groupBy $.city,
```

```
    // orderBy descending amount for leaderboard
    topOrdersSorted: (payload.orders orderBy -$.amount)
        map (o) -> { id: o.id, amount: o.amount }
}
```

Use Case: Inventory Consolidator

Scenario: Multiple warehouses report stock. Consolidate: flatten all items, remove duplicates by SKU, sort by stock level ascending (low-stock first for replenishment), and joinBy for a CSV export field.

Input

```
{
  "warehouses": [
    {"location":"WH-North",
     "stock":[{"sku":"A1","item":"Laptop","qty":5},
              {"sku":"B2","item":"Mouse","qty":120}]},
    {"location":"WH-South",
     "stock":[{"sku":"A1","item":"Laptop","qty":8},
              {"sku":"C3","item":"Keyboard","qty":35}]}
  ]
}
```

Expected Output

```
{
  "consolidated": [
    {"sku":"A1","item":"Laptop",  "qty":5},
    {"sku":"C3","item":"Keyboard","qty":35},
    {"sku":"B2","item":"Mouse",   "qty":120}
  ],
  "csvExport": "Laptop,Keyboard,Mouse"
}
```

Solution: Inventory Consolidator

```
%dw 2.0
output application/json
---
do {
    // Step 1: flatMap to get all stock items from all
        warehouses
    var allStock = payload.warehouses flatMap $.stock

    // Step 2: distinctBy SKU (keeps first occurrence)
    var uniqueStock = allStock distinctBy $.sku

    // Step 3: orderBy quantity ascending (low stock first)
```

```
    var sorted = uniqueStock orderBy $.qty
    ---
    {
        consolidated: sorted,
        csvExport:    (sorted map $.item) joinBy ","
    }
}
```

Interview Tip

Q: What is the difference between flatten and flatMap?
A: `flatten` takes an already-nested array and flattens it one level. `flatMap` applies a map function first, then flattens the result. Use `flatMap` when your `map` would produce arrays of arrays.

Q: How does groupBy work in DataWeave?
A: `groupBy` returns an object where each key is a unique value of the grouping field and each value is an array of matching elements.

Chapter 11

Aggregation Functions – reduce

11.1 What is reduce?

Think of `reduce` like a **running total on a scoreboard**. You start with an initial value (accumulator). For each element, you update the accumulator. At the end, you get the final result.

```
// Syntax
array reduce ((element, accumulator = initialValue) -> expression)

// Sum example
[10,20,30,40,50] reduce ((item, acc=0) -> acc + item)  // 150
```

Use Case: Full Order Invoice Aggregator

Scenario: Given a list of orders, calculate: total revenue, number of orders above 50000, average order value, and a summary string of all customer names.

Input

```
{
  "orders": [
    {"id":"01","customer":"Infosys","amount":200000},
    {"id":"02","customer":"Ravi",   "amount":4500},
    {"id":"03","customer":"TCS",    "amount":150000},
    {"id":"04","customer":"Wipro",  "amount":85000},
    {"id":"05","customer":"Priya",  "amount":35000}
  ]
}
```

Expected Output

```
{
  "totalRevenue":        474500,
  "orderCount":          5,
  "highValueCount":      3,
  "averageOrderValue":   94900.0,
  "maxOrderAmount":      200000,
  "minOrderAmount":      4500,
  "customerSummary":     "Infosys, Ravi, TCS, Wipro, Priya
     "
}
```

Solution: Full Order Invoice Aggregator

```
%dw 2.0
output application/json
---
do {
    var orders = payload.orders
    var count  = sizeOf(orders)

    // Total revenue using reduce
    var total = orders reduce ((o, acc=0) -> acc + o.amount)

    // Count orders above 50000
    var highCount = orders reduce ((o, acc=0) ->
        if (o.amount > 50000) acc + 1 else acc)

    // Find maximum amount
    var maxAmt = orders reduce ((o, acc=0) ->
        if (o.amount > acc) o.amount else acc)

    // Find minimum amount (start with very large number)
    var minAmt = orders reduce ((o, acc=999999999) ->
        if (o.amount < acc) o.amount else acc)

    // Build comma-separated customer string
    var custStr = orders reduce ((o, acc="") ->
        if (acc == "") o.customer
        else acc ++ ", " ++ o.customer)
    ---
    {
        totalRevenue:       total,
        orderCount:         count,
        highValueCount:     highCount,
        averageOrderValue:  total / count,
        maxOrderAmount:     maxAmt,
        minOrderAmount:     minAmt,
        customerSummary:    custStr
    }
}
```

Use Case: Student Grade Book Aggregator

Scenario: Process a list of student exam scores across multiple subjects and calculate total marks, percentage, and letter grade for each student using reduce for totals.

Input

```
{
  "students": [
    {"name":"Arjun","scores":[85,92,78,88,95]},
    {"name":"Priya","scores":[55,62,48,70,58]},
    {"name":"Kiran","scores":[95,98,92,96,99]}
  ],
  "maxPerSubject": 100
}
```

Expected Output

```
[
  {"name":"Arjun","total":438,"percentage":87.6,"grade":"
     A"},
  {"name":"Priya","total":293,"percentage":58.6,"grade":"
     C"},
  {"name":"Kiran","total":480,"percentage":96.0,"grade":"
     A+"}
]
```

Solution: Student Grade Book Aggregator

```
%dw 2.0
output application/json

fun letterGrade(pct: Number) =
    if      (pct >= 90) "A+"
    else if (pct >= 75) "A"
    else if (pct >= 60) "B"
    else if (pct >= 40) "C"
    else                "Fail"
---
payload.students map (student) ->
    do {
        var numSubjects = sizeOf(student.scores)
        var maxTotal    = numSubjects * payload.maxPerSubject
        var total       = student.scores reduce ((s, acc=0) ->
            acc + s)
        var percentage  = (total / maxTotal) * 100
        ---
        {
            name:       student.name,
            total:      total,
            percentage: percentage,
```

```
            grade:      letterGrade(percentage)
        }
    }
```

Interview Tip

Q: When would you use reduce instead of map or filter?
A: Use `reduce` when you need to collapse an array into a **single value** – sum, count, max, min, build a string, or complex aggregation. `map` always returns same-size array; `filter` returns a subset. Only `reduce` returns a completely different type.

Chapter 12

Utility Functions

12.1 ++, –, pluck, to, uuid, zip, unzip

```
%dw 2.0
output application/json
import uuid from dw::core::Crypto
---
{
    // ++ concatenates strings, arrays, objects
    strConcat:  "Hello" ++ " " ++ "World",
    arrMerge:   [1,2,3] ++ [4,5,6],              // [1,2,3,4,5,6]
    objMerge:   {a:1,b:2} ++ {b:99,c:3},          // {a:1,b:99,c:3}

    // -- removes elements or keys
    arrDiff:    [1,2,3,4,5] -- [2,4],             // [1,3,5]
    objDiff:    {name:"Ravi",city:"HYD",age:25} -- ["city"],

    // pluck: object -> array
    plucked:    {laptop:50,mouse:200} pluck (v,k) ->
                    (k as String) ++ ": " ++ v,

    // to: range
    range:      (1 to 5),                          // [1,2,3,4,5]
    reversed:   (5 to 1),                          // [5,4,3,2,1]

    // uuid
    txnId:      uuid(),

    // zip and unzip
    zipped:     zip([1,2,3], ["a","b","c"]),
    unzipped:   unzip([[1,"a"],[2,"b"],[3,"c"]])
}
```

Use Case: API Audit Trail Builder

Scenario: Build a complete audit trail entry for every API request. Generate a UUID for correlation, merge request and user metadata, remove sensitive fields, and zip event types with timestamps.

Input

```
{
  "request": {
    "method": "POST",
    "path":   "/orders",
    "body":   { "customerId": 42, "amount": 85000 }
  },
  "user": {
    "userId":   "USR-101",
    "role":     "admin",
    "password": "secret123",
    "token":    "jwt-abc"
  },
  "eventTypes": ["REQUEST","VALIDATION","DB_WRITE","
     RESPONSE"],
  "timestamps":
     ["09:00:00","09:00:01","09:00:02","09:00:03"]
}
```

Expected Output

```
{
  "correlationId": "...(uuid)...",
  "auditEntry": {
    "method":"POST","path":"/orders",
    "userId":"USR-101","role":"admin"
  },
  "timeline": [
    {"event":"REQUEST",   "time":"09:00:00"},
    {"event":"VALIDATION","time":"09:00:01"},
    {"event":"DB_WRITE",  "time":"09:00:02"},
    {"event":"RESPONSE",  "time":"09:00:03"}
  ],
  "auditFields": ["method","path","userId","role"]
}
```

Solution: API Audit Trail Builder

```
%dw 2.0
output application/json
import uuid from dw::core::Crypto
---
do {
    // Step 1: merge request info with safe user fields (
       remove secrets)
    var safeUser    = payload.user -- ["password","token"]
    var requestMeta = { method: payload.request.method,
                        path:   payload.request.path }
    var auditEntry  = requestMeta ++ safeUser

    // Step 2: zip event types with timestamps
```

```
    var zipped   = zip(payload.eventTypes, payload.timestamps)
    var timeline = zipped map (pair) -> {
        event: pair[0],
        time:  pair[1]
    }

    // Step 3: pluck keys of the merged audit entry
    var fields = keysOf(auditEntry) map ($ as String)
    ---
    {
        correlationId: uuid(),
        auditEntry:    auditEntry,
        timeline:      timeline,
        auditFields:   fields
    }
}
```

Use Case: Configuration Merger and Range Expander

Scenario: Merge default config with environment-specific overrides, generate a paginated range of record IDs, and use pluck to build a human-readable config summary.

Input

```
{
  "defaultConfig": {
    "timeout":  3000,
    "retries":  3,
    "logLevel": "INFO",
    "pageSize": 20
  },
  "envOverrides": {
    "timeout":  5000,
    "logLevel": "DEBUG"
  },
  "startPage": 1,
  "endPage":   5
}
```

Expected Output

```
{
  "mergedConfig": {
    "timeout":3000,"retries":3,"logLevel":"DEBUG","
       pageSize":20
  },
  "pageNumbers":   [1,2,3,4,5],
  "configSummary": ["timeout = 3000","retries = 3",
                    "logLevel = DEBUG","pageSize = 20"]
}
```

Solution: Configuration Merger and Range Expander

```
%dw 2.0
output application/json
---
do {
    // ++ merges objects; right side wins on key conflicts
    var merged = payload.defaultConfig ++ payload.envOverrides

    // to generates a range array
    var pages  = (payload.startPage to payload.endPage)

    // pluck converts object to array of formatted strings
    var summary = merged pluck (value, key) ->
        (key as String) ++ " = " ++ value
    ---
    {
        mergedConfig:  merged,
        pageNumbers:   pages,
        configSummary: summary
    }
}
```

Interview Tip

Q: What does the ++ operator do when merging two objects in DataWeave?
A: It merges both objects into one. If the same key exists in both, the **right-side value wins**. Example: `{a:1,b:2} ++ {b:99,c:3}` gives `{a:1,b:99,c:3}`.

Q: When would you use `zip` in integration?
A: When you receive parallel arrays from different API calls (e.g., IDs from one system, labels from another) and need to pair them element-by-element before processing.

Chapter 13

Quick Reference & Interview Cheat Sheet

13.1 Complete Function Reference

Function	Works On	Purpose
`map`	Array	Transform each element
`mapObject`	Object	Transform each key-value pair
`filter`	Array	Keep matching elements
`filterObject`	Object	Remove unwanted keys
`reduce`	Array	Aggregate to single value
`groupBy`	Array	Group into object by key
`orderBy`	Array	Sort array
`distinctBy`	Array	Remove duplicates
`flatten`	Array of Arrays	Flatten one level
`flatMap`	Array	Map then flatten
`pluck`	Object	Convert object to array
`joinBy`	Array	Array to string
`zip`	Two Arrays	Pair elements
`unzip`	Array of pairs	Split pairs
`sizeOf`	Any	Count elements
`typeOf`	Any	Get type name
`keysOf`	Object	Get all keys
`valuesOf`	Object	Get all values
`entriesOf`	Object	Get key-value pairs
`uuid`	–	Generate unique ID
`»`	Element + Array	Prepend
`«`	Array + Element	Append
`++`	String/Array/Object	Merge/Concatenate
`-`	Array/Object	Remove elements or keys

13.2 Top 15 Interview Questions and Answers

1. **What is the difference between map and mapObject?**
   map works on arrays and returns an array. mapObject works on objects and returns an object. Both transform their elements but the input/output types differ.

2. **How do you remove null values from a JSON object?**
   `payload filterObject (value) -> value != null`

3. **What is the do operator used for?**
   Creates a local variable scope within a map or complex expression, useful for intermediate calculations.

4. **How does MuleSoft retrieve auto-generated DB keys?**
   Via `payload.generatedKeys[0].GENERATED_KEY` *after a DB Insert operation.*

5. **What is Connection Pooling and why is it important?**
   Pre-creates database connections and reuses them, reducing latency and preventing resource exhaustion under high load.

6. **What is a Correlation ID?**
   A unique UUID passed across all APIs in a transaction for end-to-end tracing and debugging across logs.

7. **What is the difference between flatten and flatMap?**
   flatten flattens an already-nested array. flatMap applies a map function and then flattens the resulting arrays.

8. **How do you call a stored procedure with OUT parameters?**
   Using the DB Stored Procedure operation, declare OUT params in the input map with `inOut: "OUT"`.

9. **What is the difference between reduce and map?**
   map returns an array of the same size; reduce collapses the entire array into one aggregated value.

10. **How do you handle SQL injection in dynamic queries?**
    Always use named parameters (`:paramName`*). Never concatenate user input directly into a SQL string.*

11. **What does if-only do in DataWeave?**
    Conditionally includes a field. The field is completely absent (not null) when the condition is false.

12. **How does groupBy differ from orderBy?**
    groupBy clusters elements into keyed groups (returns an object). orderBy sorts elements (returns an array).

13. **What does ++ do when merging two objects?**
    Merges both objects. Right-side values win on key conflicts.

14. **What is pattern matching in DataWeave?**
    The match keyword evaluates a value against multiple patterns like a switch statement, supporting value and type matching.

15. **What is the difference between filter first vs map first?**
    Filter first is always better for performance – it reduces the array before transformation. Map first processes all elements unnecessarily.

13.3 Final Words

You have worked through the entire handbook – concepts, use cases, and solutions. That is everything an integration developer needs for real projects and technical interviews.

- **Practice daily** in the DataWeave Playground: `https://dataweave.mulesoft.com`
- **Build something real:** a Customer Registration API backed by MySQL
- **Read error messages carefully:** MuleSoft errors are usually very descriptive
- **Chain functions:** the real power of DataWeave is composing filter + map + reduce

And remember – always use a `WHERE` clause in your DELETE statements!

– D Charan Jeet
Salesforce MuleSoft Developer – Cognizant